

KALBEE: A Python Toolkit for Filtering, Smoothing, and Multi-Object Tracking

Le Duc Minh

minh.leduc.0210@gmail.com

Abstract—KALBEE is an open-source Python *tracking toolkit*. It gathers the Kalman filter and its nonlinear, robust, and adaptive descendants together with RTS-family smoothers, multi-target trackers (SORT-style association, JPDA, PMBM), and noise-learning routines behind one consistent `predict/update` interface, so that a single body of application code spans everything from denoising a sensor stream to tracking pedestrians in real video. The toolkit is deliberately educational as well as practical: every algorithm shares the same mathematical vocabulary, the same ready-made motion and measurement models, and the same consistency diagnostics. This paper uses KALBEE as a concrete, fully worked example of how to build such a toolkit and, more generally, of how to *engineer* numerically reliable estimation and tracking software. We (i) frame filtering, smoothing, and multi-target tracking in a single Bayesian notation so that implementation decisions map to one shared vocabulary; (ii) catalogue the recurring failure modes—loss of covariance positive-definiteness, near-singular inverses, brittle interfaces, and evaluation that cannot be reproduced—and the defenses against them; (iii) argue for a small set of design patterns (a uniform filter interface, pluggable sigma-point strategies, ready-made model builders, and a factory for switching strategies) that let a single codebase span a dozen algorithms without duplicating application code; and (iv) describe a validation methodology based on normalized-innovation and normalized-estimation error consistency statistics rather than point estimates alone. The result is a checklist, grounded in both theory and practice, for building or evaluating a filtering and tracking toolkit, illustrated end-to-end with real pedestrian footage from the MOT Challenge.

Index Terms—Kalman filter, recursive Bayesian estimation, sensor fusion, multi-object tracking, data association, tracking toolkit, numerical software, software design, Python

I. INTRODUCTION

The Kalman filter [1] is the canonical solution to online estimation of the state of a linear system from noisy measurements. Its successors—the extended Kalman filter (EKF), unscented Kalman filter (UKF) [2], cubature Kalman filter (CKF) [3], particle filters, and a long tail of robust and adaptive variants—push the same `predict/update` recursion to nonlinear dynamics, non-Gaussian noise, and hostile operating conditions. Applications include attitude determination, GPS/IMU fusion, robotics, and multi-target tracking in radar and vision.

A practitioner implementing these algorithms quickly discovers that the textbook recursion is the *easy* part. The hard problems are numerical: covariances must remain symmetric positive-definite over long horizons; innovation covariances become near-singular under poor observability; and subtle indexing conventions for block-diagonal state vectors turn correct math into silently wrong estimates. A second class

of problems is architectural: a library that exposes twenty filters but no common interface forces application code to be rewritten for each one, and a library whose interfaces are convenient only for the author’s original use case will not survive contact with real data. A third class is methodological: how do we know a filter is working at all, and how do we compare implementations honestly?

This paper organizes the accumulated answers to these questions into a coherent, largely framework-agnostic treatment, so that a reader can apply the advice to any language or library. Our contributions are:

- 1) a unified Bayesian formulation of filtering, smoothing, and multi-target tracking with consistent notation (Section II);
- 2) a taxonomy of algorithm families and the numerical/operational failure modes each one introduces (Sections III and IV);
- 3) a small set of software design patterns that have proven sufficient to keep a broad algorithm suite maintainable and uniform (Section V);
- 4) a validation and evaluation methodology grounded in filter consistency statistics and reproducible, honest benchmarks (Section VI);
- 5) two short case studies—attitude estimation from an inertial measurement unit and multi-target tracking of pedestrians in real video—that exercise the full pipeline (Section VII).

We deliberately keep the exposition framework-agnostic; where a concrete reference implementation helps, we point to KALBEE, an open-source (Apache-2.0) library written in Python and NumPy, whose public interface and tests instantiate every pattern and validation idea below.

II. PROBLEM FORMULATION

A. Recursive Bayesian filtering

Let $x_k \in \mathbb{R}^n$ be the state at discrete time k and $z_k \in \mathbb{R}^m$ a measurement. A dynamic system is described by a transition density and a measurement density,

$$x_k \sim p(x_k | x_{k-1}), \quad z_k \sim p(z_k | x_k). \quad (1)$$

Filtering maintains the posterior $p(x_k | z_{1:k})$ through two steps. The *prediction* applies the Chapman–Kolmogorov equa-

tion,

$$p(x_k | z_{1:k-1}) = \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1}, \quad (2)$$

and the *update* applies Bayes' rule,

$$p(x_k | z_{1:k}) \propto p(z_k | x_k) p(x_k | z_{1:k-1}). \quad (3)$$

Every filter in this paper is a different way of representing the two densities in (3): moments for the Kalman filter and its Gaussian variants, a particle set for sequential Monte Carlo, and so on.

For linear-Gaussian models, $x_k = Fx_{k-1} + w_k$, $z_k = Hx_k + v_k$ with $w_k \sim \mathcal{N}(0, Q)$ and $v_k \sim \mathcal{N}(0, R)$, the posterior is Gaussian and is propagated exactly by the Kalman filter,

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1}, \quad P_{k|k-1} = FP_{k-1}F^\top + Q, \quad (4)$$

$$y_k = z_k - H\hat{x}_{k|k-1}, \quad S_k = HP_{k|k-1}H^\top + R, \quad (5)$$

$$K_k = P_{k|k-1}H^\top S_k^{-1}, \quad (6)$$

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k y_k, \quad (7)$$

$$P_k = (I - K_k H)P_{k|k-1}. \quad (8)$$

The residual y_k (innovation) and its covariance S_k are central: their statistics are the primary diagnostic instrument for filter health.

B. Nonlinear and non-Gaussian extensions

When the dynamics or measurement model are nonlinear, $x_k = f(x_{k-1}) + w_k$, $z_k = h(x_k) + v_k$, no closed-form posterior exists in general. Three approximation families dominate:

- **Linearization (EKF).** Taylor-expand f and h around the current estimate and run the linear recursion with Jacobians $F_k = \partial f / \partial x$ and $H_k = \partial h / \partial x$. Cheap, but breaks down when the true posterior is not captured by the local linearization. When Jacobians are unavailable or error-prone, a central finite-difference Jacobian is a pragmatic, testable substitute.
- **Deterministic sampling (UKF, CKF).** Approximate the distribution by a small set of weighted points propagated through the exact nonlinearity—no Jacobians required. The unscented transform places $2n + 1$ sigma points and is accurate to second order; the cubature rule uses $2n$ points and scales better in high dimensions [3].
- **Stochastic sampling (particle filters).** Represent the posterior by N weighted particles, giving an asymptotically exact approximation for arbitrary, possibly multimodal, non-Gaussian posteriors. Rao–Blackwellization marginalizes any conditionally linear substate, trading particle count for closed-form Kalman updates.

C. Smoothing and parameter learning

A filter is causal; a *smoother* additionally conditions on future measurements. The Rauch–Tung–Striebel (RTS) smoother [4] runs the filter forward and a correction pass backward, and is the workhorse for offline estimation (mapping, trajectory post-processing, EM). Fixed-lag smoothing bounds the backward horizon for near-online use.

The same machinery supports *learning*: if the model structure F, H is known but the noise covariances Q, R are not, expectation–maximization fits them by alternating the forward/backward pass (E-step) with a covariance update from the smoothed sufficient statistics (M-step) [5]. A cheaper, online alternative adapts Q, R so that the innovation statistics match their theoretical distribution, which connects directly to the consistency diagnostics of Section VI.

D. Multi-object tracking

Multiple targets require not only filtering but *data association*: which measurement belongs to which track, and which measurements are clutter. The canonical lightweight design—popularized as SORT [7]—is a bank of Kalman filters plus per-frame assignment:

- 1) predict each existing track;
- 2) build a cost matrix (e.g. squared Mahalanobis distance using each track's innovation covariance) and solve the assignment with the Hungarian algorithm under a gating threshold;
- 3) update matched tracks, age out unmatched ones, and instantiate tentative new tracks for unmatched detections;
- 4) confirm a track only after n_{init} consecutive matches and delete it after N_{max} misses.

Joint probabilistic data association (JPDA) and Poisson multi-Bernoulli mixture (PMBM) filters generalize this to principled treatment of clutter and track uncertainty, at higher computational and conceptual cost. All three share the same building blocks: per-target filters, an innovation covariance, and a gate. This is why a filtering library and a tracking library are naturally two layers of the same design rather than separate products.

III. AN ALGORITHMIC TAXONOMY

A library author must decide *which* algorithms to implement and, more importantly, how a user should choose among them. The decision tree that has proved durable in practice is:

- **Linear, Gaussian** → Kalman filter. Use the square-root or information form when numerics or many sensors demand it.
- **Nonlinear** → EKF if Jacobians are easy and the nonlinearity is mild; otherwise UKF/CKF (no derivatives).
- **Non-Gaussian / multimodal** → particle filter; Rao–Blackwellized particle filter if the model is partially linear.
- **Maneuvering targets** → interacting multiple models (IMM): run several filters with different process models in parallel and blend them by model likelihood. A cheap alternative is a fading-memory filter that inflates the predicted covariance by a factor > 1 to discount stale data.
- **Unknown noise statistics** → adaptive filtering, EM, or innovation-based auto-tuning.
- **Multiple targets** → per-target filters plus an association layer (SORT/JPDA/PMBM).

- **Adversarial or uncertain models** $\rightarrow H_\infty$ filtering, which minimizes worst-case estimation error rather than assuming known noise.

The critical insight for software design is that nearly all of these are variations on the same `predict/update` contract; only the innards differ. That observation, more than any individual algorithm, drives the architecture of Section V.

IV. ENGINEERING A FILTER IS NOT THE SAME AS WRITING ONE

Five concerns separate a working demo from software that survives deployment.

A. Numerical stability

The textbook update $P_k = (I - KH)P_{k-1}$ is algebraically correct but numerically fragile: round-off can make P lose symmetry and, worse, positive-definiteness, after which a Cholesky factorization or a log-likelihood evaluation fails or silently produces nonsense. Durable defenses, in increasing order of sophistication, are:

- 1) the Joseph form $P_k = (I - KH)P_{k-1}(I - KH)^\top + K R K^\top$, which is guaranteed positive semi-definite for any gain;
- 2) explicit symmetrization $P \leftarrow (P + P^\top)/2$ after every covariance update;
- 3) *condition-aware* inversion. Plain `inv( $S$ )` returns garbage without raising for near-singular S ; checking the condition number and falling back to a regularized or pseudo-inverse gives predictable behavior instead of silent corruption;
- 4) square-root forms that propagate the Cholesky factor of P directly, keeping the covariance implicitly positive-definite by construction.

A reference implementation should apply these policies *uniformly* through one shared linear-algebra helper rather than ad hoc in each filter.

B. Determinism and reproducibility

Filters are increasingly used inside evaluation harnesses and learned pipelines. Seedable, isolated random streams (per-constructor random number generators that do not perturb global state) make particle and ensemble methods reproducible; giving every algorithmic constant a single home (one module-level constants registry) makes tuning repeatable.

C. State conventions

Batched or multi-axis states invite ordering bugs. A library that fixes one convention—e.g. block-per-axis layout “[`axis1, v1, axis2, v2, ...`]”—and expresses all model builders (“constant velocity”, “constant turn”, “measurement of position”) through that convention eliminates an entire class of silent errors and makes the corresponding tests trivial to write.

D. Irregular data

Real streams drop measurements, arrive asynchronously, or arrive out of order. A filter API should therefore treat “predict-only” steps as first-class (no measurement this tick), support batch processing over sequences with an explicit “missing” sentinel, and offer an out-of-sequence-measurement buffer for multi-rate sensors.

E. Optional dependencies

A modern estimation library will want `DataFrame` and `scikit-learn` integrations, terminal demos, plotting, and (sometimes) `PyTorch`. None of these belong in the core dependency set; the core should import only the numerics stack, with integrations loaded lazily or behind extras.

V. DESIGN PATTERNS

The following five patterns, each simple in isolation, compose into a library that can host a dozen algorithms without forking application code.

A. A uniform filter interface

Define one abstract filter with the two essential operations and a handful of secondary conveniences,

```
class Filter(ABC):
    def predict(self, dt=1.0, **kw): ...
    def update(self, measurement, **kw): ...
    def measure(self, state=None): ... # state -> measurement space
    def predict_only(self, dt=1.0, **kw): ...
    def reset(self, state=None, covariance=None): ...
    def filter_sequence(self, measurements, dt=1.0, missing=None): ...
```

Because every filter—linear, nonlinear, particle-based, or information form—obeys the same contract, a user can swap a Kalman filter for an adaptive or square-root variant by changing one line, and the entire higher-level machinery (tracking, tuning, `DataFrame` integration, diagnostics) works unchanged.

B. Strategy objects for pluggable internals

Hard-coding one sigma-point rule inside the UKF prevents experimentation. Factoring the rule into a small strategy object (`SimplexSigmaPoints`, `MerweScaledSigmaPoints`, `JulierSigmaPoints`) lets a single filter implementation expose every variant through constructor injection. The same idea recurs for measurement gates and association algorithms.

C. Ready-made model builders

Users should not re-derive F , Q , H , R by hand for the standard motion/measurement families. A small set of parameterized builders (constant-velocity, constant-acceleration, coordinated-turn, position measurement) returns the matrices consistent with the library’s state convention, so a “correct” model is obtained by construction and every documented example shares it.

D. A factory for strategy selection

String-based construction (`mode="ekf"`, `mode="ukf"`, ...) lets configuration files, CLIs, and hyperparameter sweeps choose filters without importing them. It also gives integration layers (a scikit-learn-style estimator, a command-line benchmark) a single entry point.

E. Procedural alongside object-oriented

Many users come from FilterPy-style code and want functions (`predict(x, P, F, Q)` returning the new pair) as much as stateful objects. Exposing both on top of the same math is cheap and dramatically widens adoption; one should be a thin wrapper over the other so they cannot drift apart.

F. Batching as an implementation detail

When N independent series must be filtered (many tracked objects, many replicates), a naive per-series loop wastes the vectorized hardware. A *vectorized filter* that broadcasts every matrix operation over a leading batch axis delivers large speedups while preserving the uniform interface—users switch `KalmanFilter` to a batched subclass, not to a separate library.

VI. VALIDATION AND EVALUATION

A. Consistency statistics beat point estimates

A single RMSE number cannot distinguish a well-tuned filter from a lucky one. The standard theory-backed diagnostics are the normalized innovation squared (NIS) and normalized estimation error squared (NEES). For a consistent filter, each is chi-squared distributed with, respectively, m and n degrees of freedom [6]:

$$\text{NIS}_k = y_k^\top S_k^{-1} y_k \sim \chi_m^2, \quad \text{NEES}_k = e_k^\top P_k^{-1} e_k \sim \chi_n^2, \quad (9)$$

where $e_k = x_k - \hat{x}_k$ uses the true state (available in simulation or with ground truth). Averaging NIS/NEES over a run and comparing against the expected value detects over- and under-confidence; the innovation autocorrelation (whiteness test) detects model mismatch. Consistency tests should be first-class library functions, because they are exactly what a developer needs when tuning Q and R —and they are the theoretical basis for the innovation-based auto-tuners of Section II.

B. Reproducible, honest benchmarks

Benchmarks are only useful if the task, the data, and the reference implementations are public and the results are reported without cherry-picking. Concretely: fix one task (same F, Q, H, R , same measurement sequence), compare against reference libraries on identical inputs, and report *both* the case the new library wins and the case it loses. In our reference suite, a purpose-built vectorized filter outperforms a dedicated batching library by roughly $4\times$ on 1,000 concurrent series, while a minimal single-purpose library still has lower per-call overhead for one bare filter loop—both results are worth stating, because they tell different users different truths.

C. Testing the math

Regression tests should include cross-checks between algebraically equivalent implementations (e.g. square-root vs. standard Kalman filter must agree to machine precision), closed-form one-step numerical answers, analytic vs. finite-difference Jacobians, and stress cases (zero time step, extreme noise, exactly singular innovation covariance). These tests are the real documentation of numerical policy.

VII. CASE STUDIES

A. Attitude estimation from an inertial measurement unit

A common, instructive problem: estimate orientation from a gyroscope (process: integrate angular velocity) corrected by an accelerometer (measurement: the direction of gravity in the body frame). With a unit quaternion $q = [w, x, y, z]^\top$ as the state, a constant-angular-velocity step integrates to a delta quaternion and the process model is the quaternion product, which is *linear* in the state—its Jacobian is the exact right-multiplication matrix of the delta quaternion, not an approximation. Two lessons recur. First, observability: gravity alone cannot observe yaw; without a magnetometer or course measurement, yaw noise random-walks uncorrected. Second, naive quaternion-state EKFs linearize only locally, so over-aggressive measurement noise assumptions cause the corrected state to leave the unit manifold before renormalization; the documented remedy (inflate the assumed R modestly) is precisely the kind of hard-won tuning knowledge that belongs in a library's documentation and tests, not in the head of its author. A general treatment of quaternion kinematics is given in [8].

B. Pedestrian multi-target tracking in real video

To exercise the whole stack, we track pedestrians in the public MOT16 benchmark sequence MOT16-02 [9]: each ground-truth box $[x_1, y_1, x_2, y_2]$ is treated as a four-axis measurement and fed to the per-corner constant-velocity bank with gated Hungarian association. The tracker keeps stable identities through brief occlusions and confirms tracks only after consecutive detections; overlaying the filtered boxes on the real frames makes association behavior legible in a way that synthetic data cannot. The same tracker, fed by a YOLO detector instead of ground truth, becomes a real-time vehicle/person tracker; the only change is the source of the boxes. This underlines the layering argument of Section II: the filtering core never needs to know what produced the measurements.

VIII. RELATED WORK

a) *FilterPy*: provides pedagogical, textbook-faithful implementations of the KF/EKF/UKF family and an RTS smoother, with no shared abstract base across all filters and little recent maintenance. It remains the clearest reference for learning the equations.

b) *pykalman*: offers a compact filter/smooth/EM interface in the scikit-learn idiom, restricted to the linear-Gaussian model.

c) *simdkalman*: vectorizes the linear KF across a leading axis—a narrow, well-executed contribution with no nonlinear or tracking support.

d) *Stone Soup*: is a component-based research framework from the defence community covering the full tracking pipeline (hypothesisers, associators, metrics) at a much higher conceptual cost than a library whose users mostly need “fuse a GPS fix with an accelerometer.”

e) *Differentiable filters*: (e.g. JAX-based Kalman smoothing) target learning-based pipelines and assume the user adopts the autodiff ecosystem wholesale.

The common thread relevant to this paper is that none of these systems treats numerical-stability policy, consistency diagnostics, and interface uniformity as parts of the same design problem—which is the gap this paper addresses.

IX. DISCUSSION AND CONCLUSION

Recursive estimation is easy to implement and hard to operationalize. This paper has argued that the hard parts are systematically addressable: a uniform Bayesian formulation keeps the algorithm space navigable; condition-aware numerics and square-root forms prevent the silent failures that plague naive implementations; five modest design patterns keep a broad algorithm suite maintainable; and consistency statistics, rather than point metrics, provide the feedback loop that makes tuning and evaluation scientific. We have illustrated each point with a freely available reference implementation (the KALBEE library), whose public interface, model builders, and test suite instantiate the advice above and whose MOT16 case study runs on real pedestrian footage.

Two limitations deserve restatement. First, a deterministic sigma-point/particle-free stack has no GPU/autodiff backend; gradient-based noise learning and large-scale parallel filtering remain a separate, parallel implementation surface. Second, strict static typing across a long-lived numerical codebase—particularly narrowing `Optional[np.ndarray]` attributes on the base filter—is valuable but must be retrofitted incrementally rather than assumed. We consider both honest statements of scope rather than defects.

AVAILABILITY

The reference implementation used throughout is released under the Apache License 2.0, installable from PyPI and available at <https://github.com/LakoreAI/kalbee>, with documentation at <https://lakoreai.github.io/kalbee>. Data for the pedestrian case study is from the MOT Challenge [9] (<https://motchallenge.net>); a small fetcher script downloads only the needed sequence.

REFERENCES

- [1] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] S. J. Julier and J. K. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [3] I. Arasaratnam and S. Haykin, “Cubature Kalman filters,” *IEEE Transactions on Automatic Control*, vol. 54, no. 6, pp. 1254–1269, 2009.
- [4] H. E. Rauch, F. Tung, and C. T. Striebel, “Maximum likelihood estimates of linear dynamic systems,” *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965.
- [5] R. H. Shumway and D. S. Stoffer, “An approach to time series smoothing and forecasting using the EM algorithm,” *Journal of Time Series Analysis*, vol. 3, no. 4, pp. 253–264, 1982.
- [6] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.
- [7] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [8] J. Solà, “Quaternion kinematics for the error-state Kalman filter,” *arXiv preprint arXiv:1711.02508*, 2017.
- [9] A. Dendorfer *et al.*, “MOT16: A benchmark for multi-object tracking,” *arXiv preprint arXiv:1603.00831*, 2016.
- [10] R. Labbe, “FilterPy: Python Kalman filtering and optimal estimation library,” GitHub repository. [Online]. Available: <https://github.com/rlabbe/filterpy>
- [11] pykalman contributors, “pykalman: Kalman Filter, Smoother, and EM Algorithm for Python,” GitHub repository. [Online]. Available: <https://github.com/pykalman/pykalman>
- [12] O. Seiskari, “simdkalman: Python Kalman filters vectorized as SIMD,” GitHub repository. [Online]. Available: <https://github.com/oseiskar/simdkalman>
- [13] Defence Science and Technology Laboratory (Dstl), “Stone Soup,” GitHub repository. [Online]. Available: <https://github.com/dstl/Stone-Soup>
- [14] AaltoML, “kalman-jax: Approximate inference for Markov Gaussian processes using iterated Kalman smoothing, in JAX,” GitHub repository. [Online]. Available: <https://github.com/AaltoML/kalman-jax>
- [15] L. D. Minh, “KALBEE: a clean, modular implementation of Kalman filters and estimation algorithms,” v1.0.0, 2026. [Online]. Available: <https://github.com/LakoreAI/kalbee>